

practical no-01

Aim: Implementation and time complexity of basic sorting algorithm.

Theory: Bubble sort is the simplest sorting algorithm that works by repeatedly swapping the adjacent elements if they are in the wrong order. This algorithm is not efficient for large data sets as its average and worst-case time complexity are quite high.

ex: | 5 | 6 | 1 | 3 | $i = 0$

| 5 | 6 | 1 | 3 | $i = 1$

| 5 | 1 | 6 | 3 | $i = 2$

| 5 | 1 | 3 | 6 | $i = 3$

| 1 | 5 | 3 | 6 | $i = 4$

| 1 | 3 | 5 | 6 | $i = 5 \rightarrow$ sorted element.

ARISE & SHINE

Complexity Analysis of bubble sort.

(1) Time complexity: $O(n^2)$

(2) Auxiliary space: $O(1)$

a) Insertion sort: Insertion sort is a simple sorting algorithm that works by iteratively inserting each element of an unsorted list

into its correct position in a sorted portion of the list. It is like sorting playing cards in your hands.

Ex: key = 11

arr [] =

12	11	13	5	6
11	12	13	5	6
11	12	13	5	6
5	11	12	13	6
5	6	11	12	13

arr [] =

5	6	11	12	13
---	---	----	----	----

 → sorted.

Time Complexity

Case	Time
(i) Best	$O(n)$
(ii) Average	$O(n^2)$
(iii) Worst	$O(n^2)$

- Fast for small or nearly sorted lists
- Easy to implement.

(3) Selection sort

Find the smallest element. place it at the

program: Bubble sort

```
# include <stdio.h>
```

```
int bubbleSort (int arr[], int n) {
```

```
    int i, j, temp;
```

```
    int swaps = 0;
```

```
    for (i = 0; i < n - 1; i++) {
```

```
        for (j = 0; j < n - i - 1; j++)
```

```
            if (arr[j] > arr[j + 1]) {
```

```
                temp = arr[j];
```

```
                arr[j] = arr[j + 1];
```

```
                arr[j + 1] = temp;
```

```
                swaps++;
```

```
            }
```

```
        }
```

```
    }
```

```
    return swaps;
```

```
}
```

// Function to print the array

```
void printArray (int arr[], int n) {
```

```
    for (int i = 0; i < n; i++) {
```

```
        printf ("%d ", arr[i]);
```

```
    }
```

```
printf ("\n");
```

```
int main () {
```

```
    int n;
```

```
    scanf ("%d", &n);
```

```
    int arr[n];
```

```
    printArray (arr, n);
```

```
    printf ("%d\n", totalSwaps);
```

```
    return 0;
```

```
}
```

program : insertion sort

include <stdio.h>

void insertionSort (int a[], int n, int shift) {

for (int i = 1; i < n; i++) {

int key = a[i];

int key = j = 1;

while (j > 0 && a[j] > key) {

a[j+1] = a[j];

j--;

(* shift)++;

}

a[j+1] = key;

}

int main () {

int n;

scanf ("%d", &n);

int a[n], shift = 0;

for (int i = 0; i < n; i++)

scanf ("%d", &a[i]);

insertionSort (a, n, &shift);

for (int i = 0; i < n; i++)

printf ("%d", a[i]);

printf ("\n %d", shift);

return 0;

}

program : selection sort.

include <stdio.h>

int main () {

int n, i, j, maxIndex, temp;

int arr[100];

scanf ("%d", &n);

for (i = 0; i < n; i++) {

scanf ("%d", &arr[i]);

}

for (i = 0; i < n-1; i++) {

maxIndex = i;

for (j = i+1; j < n; j++) {

if (arr[j] > arr[maxIndex])

maxIndex = j;

}

}

temp = arr[i];

arr[i] = arr[maxIndex];

arr[maxIndex] = temp;

}

for (i = 0; i < n; i++) {

printf ("%d", arr[i]);

}

return 0;

}

beginning, Repeat for remaining list.

ex: [64, 25, 12]

smallest is 12 $\rightarrow$ swap $\rightarrow$ [12, 25, 64]

time complexity.

(i) Best $- O(n^2)$

(ii) Average $- O(n^2)$

(iii) worst $- O(n^2)$

Conclusion: Hence we proved implementation and time complexity is of basic algorithm.

$$\frac{3}{3} + \frac{6}{1} + \frac{4}{5} = \frac{13}{5}$$

(MRS) 2
13-1-26

Date: 13/01/26
Day: Tuesday

Practical no-02

Aim: Implementation and analysis of advance sorting algorithm - (1) Merge sort (2) Quick sort (Comparison of divide & conquer technique).

Theory: Advanced sorting algorithms are faster and more efficient than basic sorting methods like bubble, selection, and insertion sort. They use a powerful approach called divide and conquer.

1. Merge sort: Merge sort is a divide and conquer sorting algorithm that divides the array into two equal halves, sort each half, merges the sorted halves to get a final sorted array.

ex: [38, 27, 43, 3] [9, 82, 10]

[38, 27] [43, 3] [9, 82] [10]

[27, 38] [3, 43] [9, 82] [10]

[3, 27, 38, 43] [9, 10, 82]

[3, 9, 10, 27, 38, 43, 82]

Merge Sort Analysis.

Best Case - $O(n \log n)$

Average case - $O(n \log n)$

Worst case - $O(n \log n)$

Extra space needed + yes

Stable sort - yes

Used when - large data, external sorting

2] Quick sort - Quick sort is also a divide and conquer algorithm, it selects a pivot element, places pivot in the correct position.

Divides array into: left side - smaller elements
Right side - larger elements.

ex: Array: [10, 7, 8, 9, 1, 5]

Pivot = 5

Rearrange: [1] 5 [10, 7, 8, 9]

[1] 5 [7, 8, 9, 10]

final - [1, 5, 7, 8, 9, 10]

Quick sort Analysis:

Best case = $O(n \log n)$

Average case - $O(n \log n)$

Worst case - $O(n^2)$

Extra space needed - NO

Stable sort - NO

Fastest in practice.

Explorer

C merge.c

```

1  #include <stdio.h>
2  #include <stdlib.h>
3
4  void merge(int arr[], int left, int mid, int right) {
5
6      int n1 = mid - left + 1;
7      int n2 = right - mid;
8
9      int L[n1], R[n2];
10
11     for (int i = 0; i < n1; i++)
12         L[i] = arr[left + i];
13     for (int j = 0; j < n2; j++)
14         R[j] = arr[mid + 1 + j];
15
16     int i = 0, j = 0, k = left;
17
18
19     while (i < n1 && j < n2) {
20         if (L[i] <= R[j]) {
21             arr[k] = L[i];
22             i++;
23         } else {
24             arr[k] = R[j];
25         }

```

Submit

Debugger

C merge.c

```

54     while (i < n1) {
55         arr[k] = L[i];
56         i++;
57         k++;
58     }
59
60
61     while (j < n2) {
62         arr[k] = R[j];
63         j++;
64         k++;
65     }
66
67 }
68
69 void mergeSort(int arr[], int left, int right) {
70
71     if (left < right) {
72         int mid = left + (right - left) / 2;
73
74         mergeSort(arr, left, mid);
75         mergeSort(arr, mid + 1, right);
76
77         merge(arr, left, mid, right);
78
79     }

```

Submit

Debugger

```

77         merge(arr, left, mid, right);
78     }
79
80
81
82 }
83
84 int main() {
85     int n;
86     scanf("%d", &n);
87
88     int *arr = (int *)malloc(n * sizeof(int));
89     for (int i = 0; i < n; i++) {
90         scanf("%d", &arr[i]);
91     }
92
93     mergeSort(arr, 0, n - 1);
94
95     for (int i = 0; i < n; i++) {
96         printf("%d ", arr[i]);
97     }
98
99     free(arr);
100    return 0;
101 }

```

LAUNCH

Average time

0.066 s

65.60 ms

Maximum time

0.086 s

86.00 ms

3 out of 3 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1 86 ms

Expected output

5

9 -5 7 -2 -1

-5 -2 -1 7 9

Actual output

5

9 -5 7 -2 -1

-5 -2 -1 7 9

Debug

Test case 2 71 ms

Expected output

7

2 9 1 6 3 10 7

1 2 3 6 7 9 10

Actual output

7

2 9 1 6 3 10 7

1 2 3 6 7 9 10

Debug

Test case 3 47 ms

Expected output

5

3 7 4 1 5

1 3 4 5 7

Actual output

5

3 7 4 1 5

1 3 4 5 7

Debug

C quick.c

```
27     if (low < high) {
28         int pi = partition(arr, low, high);
29         quickSort(arr, low, pi - 1);
30         quickSort(arr, pi + 1, high);
31     }
32
33 }
34
35 int main() {
36     int n;
37     scanf("%d", &n); // Read array size
38
39     int arr[n];
40     for (int i = 0; i < n; i++) {
41         scanf("%d", &arr[i]); // Read array elements
42     }
43
44     quickSort(arr, 0, n - 1);
45
46     for (int i = 0; i < n; i++) {
47         printf("%d ", arr[i]);
48     }
49
50     return 0;
51 }
```

Submit

C quick.c

```
1  #include <stdio.h>
2
3  void swap(int *a, int *b) {
4      int temp = *a;
5      *a = *b;
6      *b = temp;
7  }
8
9  int partition(int arr[], int low, int high) {
10     int pivot = arr[high];
11     int i = low - 1;
12     for (int j = low; j < high; j++) {
13         if (arr[j] < pivot) {
14             i++;
15             swap(&arr[i], &arr[j]);
16         }
17     }
18     swap(&arr[i + 1], &arr[high]);
19     return i + 1;
20 }
21
22 void quickSort(int arr[], int low, int high) {
23
24
25 }
```

Submit

Explorer

0.065 s
65.25 ms



0.077 s
77.00 ms



2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1

Expected output

5

1 4 9 3 6

1 3 4 6 9

Actual output

5

1 4 9 3 6

1 3 4 6 9

Debug



Test case 2

Expected output

6

-35 -29 36 42 20 68

-29 -35 20 36 42 68

Actual output

6

-15 -29 36 42 20 68

-29 -15 20 36 42 68

Debug



Comparison: merge sort vs quick sort.

Feature	merge sort	quick sort.
Divide method	Always divide into halves	Divide using pivot
Combine step	Required (merge)	not required
worst case	$O(n \log n)$	$O(n^2)$
Extra space	Yes	No
Stability	Stable	Not stable
speed	slower	Faster
used in	linked list, large data	Arrays, system.

Both merge sort and quick sort use divide and conquer. merge sort is safe and stable. Quick sort is faster and memory efficient. These algorithms are widely used in real-world systems.

Conclusion: Hence, we studied the implementation and analysis of advance sorting algorithm of merge sort and quick sort.

$$\frac{3}{3} + \frac{5}{7} + \frac{4}{5} = \frac{12}{15}$$

Date: 20-01-25

Day: Tuesday

Practical No-03

Aim: Implementation of searching techniques.

1. Linear search
2. Binary search (comparison of their time complexities on various inputs)

Theory: Linear search: Linear search is the simplest searching method. It checks each element one by one until the required element is found or the list ends.

- # How it works:-
1. Starts from the first element
 2. Compares it with search value.
 3. If not equal, moves to the next element.
 4. Continue until the element is found or list ends.

ex: Array: 10 25 30 45 60

search element: 45

Comparison order: 10 $\rightarrow$ 25 $\rightarrow$ 30 $\rightarrow$ 45

time complexity of linear search.

- (1) Best case: $O(1)$ $\rightarrow$ element found at first position.
- (2) Worst case: $O(n)$ $\rightarrow$ element at last position or not present.
- (3) Average case: $O(n)$

2] Binary Search: Binary search is a fast searching method, but it works only on sorted array.

How it works:

1. find the middle element.
2. If middle element = search value $\rightarrow$ found
3. If search value < middle $\rightarrow$ search left half.
4. If search value > middle $\rightarrow$ search right half.
5. Repeat until found or range ends.

ex: sorted array: 10 20 30 40 50 60
search element: 40

steps: middle: 30
search right half
middle: 40 (found).

time complexity of binary search.

- (1) Best case: $O(1)$ $\rightarrow$ element found at middle.
- (2) Worst case: $O(\log n)$
- (3) Average case: $O(\log n)$

Average time

0.069 s

68.75 ms

Maximum time

0.100 s

100.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1 100ms

Expected output

```
3
101 Alice
103 Alex
102 Amar
103
Student Found: 103 Alex
```

Debug

Actual Output

```
3
101 Alice
103 Alex
102 Amar
103
Student Found: 103 Alex
```

Test case 2 31ms

Expected output

```
4
5 Amar
6 Bob
2 Alex
1 Samuel
3
Student not found
```

Debug

Actual output

```
4
5 Amar
6 Bob
2 Alex
1 Samuel
3
Student not found
```

Submit

C binary.c

```
1  #include <stdio.h>
2  #include <string.h>
3
4  struct Student {
5      //write your code here..
6      int id;
7      char name[50];
8  };
9
10 /* Function to sort objects based on id (Bubble Sort) */
11 void sort( struct Student s[], int n ) {
12     //write your code here...
13
14     struct Student temp;
15     for (int i = 0; i < n - 1; i++) {
16         for (int j = 0; j < n - i - 1; j++) {
17             if (s[j].id > s[j + 1].id) {
18                 temp = s[j];
19                 s[j] = s[j + 1];
20                 s[j + 1] = temp;
21             }
22         }
23     }
24
25 }
```


C binary.c

/* Function to perform Binary Search on objects */
int binarySearch(struct Student s[], int n, int key) {
 int low = 0, high = n - 1, mid;

while (low <= high) {
 mid = (low + high) / 2;

if (s[mid].id == key)

return mid;

else if (s[mid].id < key)

low = mid + 1;

else

high = mid - 1;

return -1;

int main() {

int n, key;

struct Student s[100];

scanf("%d", &n);

for (int i = 0; i < n; i++) {

scanf("%d %s", &s[i].id, s[i].name);

int n, key;

struct Student s[100];

scanf("%d", &n);

for (int i = 0; i < n; i++) {

scanf("%d %s", &s[i].id, s[i].name);

scanf("%d", &key);

/* Sort objects before binary search */
sort(s, n);

int result = binarySearch(s, n, key);

if (result != -1) {

printf("Object found: ID = %d, Name = %s\n", s[result].id,
s[result].name);

} else {

printf("Object not found\n");

return 0;

Average time

0.077 s

76.75 ms

Maximum time

0.093 s

93.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1 74 ms

Expected output

3

101 Alice

207 Bob

508 Alex

706

Object not found

Actual output

3

101 Alice

207 Bob

508 Alex

706

Object not found

Test case 2 83 ms

Expected output

4

1 Bob

8 Evan

9 Diana

5 Charlie

8

Object found: ID = 8, Name = Evan

Actual output

4

1 Bob

8 Evan

9 Diana

5 Charlie

8

Object found: ID = 8, Name = Evan

Submit

```

C linear.c
1  #include <stdio.h>
2
3  struct Student {
4      int id;
5      char name[50];
6  };
7
8  int linearSearch(struct Student s[], int n, int searchID) {
9      //write your code here...
10     for (int i = 0; i < n; i++) {
11         if (s[i].id == searchID) {
12             return i;
13         }
14     }
15
16     return -1;
17 }
18
19 int main() {
20     int n, key;
21     scanf("%d", &n);
22
23     struct Student s[n];
24     for (int i = 0; i < n; i++) {
25         scanf("%d %s", &s[i].id, s[i].name);
26     }
27
28     scanf("%d", &key);
29
30     int pos = linearSearch(s, n, key);
31
32     if (pos != -1)
33         printf("Student found: %d %s", s[pos].id, s[pos].name);
34     else
35         printf("Student not found");
36
37     return 0;
38 }

```

• hi
 cl
 • B
 cl
 • fo
 • fo
 fear
 Arra
 me
 Bes
 wor
 spe
 Easy
 Com
 of

- Linear search is simple but slow for large data.
- Binary search is faster but needs sorted data.
- for small or unsorted data $\rightarrow$ linear search.
- for large and sorted data $\rightarrow$ binary search.

Feature	Linear search	Binary search
Array type	sorted / unsorted	sorted only
Method	one by one	divide & search
Best case	$O(1)$	$O(1)$
Worst case	$O(n)$	$O(\log n)$
Speed	slow	fast
Easy to implement	Yes	moderate.

Conclusion: Hence, we studied the implementation of both linear search and binary search.

$$\frac{3}{3} + \frac{7}{7} + \frac{3}{5} = \frac{13}{5}$$

MRA
27-1-26

Aim: Job sequencing with deadlines using greedy approach (profit maximization + complexity evaluation).

Theory: You are given n jobs, where each job i has:

- a deadline d_i
- a profit p_i
- each job takes exactly 1 unit of time.

Greedy strategy - The greedy approach works:

- (1) High profit jobs are more valuable.
- (2) Jobs with later deadlines offer more complexity.

Greedy choice

1. Sort all jobs in descending order of profit.
2. Schedule each job as late as possible before its deadline.

Algorithm (step-by-step)

1. Sort jobs in decreasing order of profit.
2. Create a slot array of size $\max(\text{deadline})$.
3. Consider array, with maximum size of a deadline, array of a maximum deadline.

Ex: $J_1 \rightarrow D=2, P=103$

$J_2 \rightarrow D=1, P=27$

$J_3 \rightarrow D=2, P=50$

$J_4 \rightarrow D=5, P=45$

1. Jobs $\rightarrow$ Deadline and profit

2. Sorting $\rightarrow J_1 \rightarrow J_3 \rightarrow J_4 \rightarrow J_2$

3. Array with size of max D

J_1	J_1	J_3	J_3	J_2
-------	-------	-------	-------	-------

Time complexities

Step 1: Sorting jobs - $O(n \log n)$

Step 2: Slot allocation - $O(n \times D)$ worst case

Step 3: Overall - $O(n \log n + nD)$

~~##~~ Correctness Intuition

ARISE & SHINE

(1) If a job with higher profit is excluded in favor of a lower-profit job, swapping them increases total profit.

(2) Scheduling jobs as late as possible leaves room for jobs with earlier deadlines.

Aim:

Given a set of n jobs where each job j has a deadline and profit associated with it. Each job takes 1 unit of time to complete, and only one job can be scheduled at a time. We earn the profit associated with the job if and only if the job is completed by its deadline.

Find the number of jobs done and the maximum profit.

Input Format:

- First line: An integer N representing the number of jobs.
- Second line: N integers separated by spaces, where each set of three integers represents a job in the format: Job ID & Job deadline & Job profit

Output Format:

- Two integers separated by a space: The first integer represents the number of jobs done. The second integer represents the maximum profit obtained.

Example 1:

Input:

```
4 1 4 20 2 1 10 3 1 40 4 1 30
```

Output:

```
2 60
```

Explanation: Job 1 and Job 3 can be done with a maximum profit of 60 (20+40).

Example 2:

Input:

```
5 1 2 100 2 1 19 3 2 27 4 1 25 5 1 15
```

Output:

```
2 127
```

Explanation: 2 jobs can be done with a maximum profit of 127 (100+27).

Note: Refer to visible test cases for better clarification.

Jobs will be given in the form (Jobid, Deadline, Profit) associated with that Job. The deadline of the job is the time by which the job needs to be completed to earn a profit.

Constraints:

- $1 \leq N \leq 10^5$
- $1 \leq \text{Deadline} \leq N$
- $1 \leq \text{Profit} \leq 500$

Source Code:

CTC31044.c

```
#include <stdio.h>
#include <stdlib.h>

struct Job {
    int id;
    int deadline;
    int profit;
};

// Comparator function to sort jobs by profit (descending)
int compare(const void *a, const void *b) {
    struct Job *j1 = (struct Job *)a;
    struct Job *j2 = (struct Job *)b;
    return j2->profit - j1->profit;
}

int main() {
    int n;
    scanf("%d", &n);

    struct Job jobs[n];
```

```

for (int i = 0; i < n; i++) {
    scanf("%d %d %d", &jobs[i].id, &jobs[i].deadline, &jobs[i].profit);
}

// Sort jobs by profit
qsort(jobs, n, sizeof(struct Job), compare);

// Find maximum deadline
int maxDeadline = 0;
for (int i = 0; i < n; i++) {
    if (jobs[i].deadline > maxDeadline)
        maxDeadline = jobs[i].deadline;
}

// Slot array to keep track of occupied time slots
int slot[maxDeadline];
for (int i = 0; i < maxDeadline; i++)
    slot[i] = -1;

int jobCount = 0, maxProfit = 0;

// Schedule jobs
for (int i = 0; i < n; i++) {
    for (int j = jobs[i].deadline - 1; j >= 0; j--) {
        if (slot[j] == -1) {
            slot[j] = jobs[i].id;
            jobCount++;
            maxProfit += jobs[i].profit;
            break;
        }
    }
}

printf("%d %d", jobCount, maxProfit);

return 0;
}

```

Execution Results - All test cases have succeeded!

User Output

4

1 4 20 2 1 10 3 1 40 4 1 30

2 60

Test Case - 1

User Output

5

1 2 100 2 1 19 3 2 27 4 1 25 5 1 15

2 127

Test Case - 2

3. This satisfies both: (1) Greedy-choice property
2. optimal substructure.

Time Complexity Analysis:-

(1) n = number of jobs

(2) D = maximum deadlines.

sorting jobs - $O(n \log n)$

slot allocator - $O(n \times D)$ (worst case)

overall - $O(n \log n + nD)$

Conclusion : Hence, all scheduled job sequencing with deadlines using greedy approach (profit maximization + complexity evaluation)

$$\frac{3}{3} + \frac{4}{7} + \frac{4}{5} = \frac{14}{15}$$

WRA &
30-1-26

Aim: Fractional knapsack problem using greedy method (efficiency comparison using different item set)

Theory: What is the knapsack problem $\rightarrow$

- A bag (knapsack) with fixed capacity
- Some items, each having $\rightarrow$ weight, profit (value)

Goal: put items in the bag to get maximum profit without exceeding capacity.

2] What is fractional knapsack $\rightarrow$ you can take a fraction (part) of an item (like half apple, not whole only)

Ex: If item weight = 10 kg and bag has space for 5 kg $\rightarrow$ you can take 50% of item.

This is different from 0/1 knapsack, where items must be taken fully or not at all.

Why greedy method is used $\rightarrow$ At each step, take the best possible choice right now.

For fractional knapsack, the best choice is:
Take the item with maximum profit per unit weight. Reason: Higher profit per kg = more profit in less space.

- # 1. Calculate Profit/weight ratio for each item.
2. Sort items in descending order of this ratio.
3. Start adding items:- ① If full item fits $\rightarrow$ take it.
② Else $\rightarrow$ take any required fraction.
4. Stop when knapsack is full

Algorithm.

Input: n items, each with profit and weight
Knapsack Capacity W

Step 1: For each item, calculate profit/weight ratio.

Step 2: Sort item in decreasing order of ratio.

Step 3: total Profit = 0

Step 4: For each item:

If item weight $\leq W$:

Take whole item

$W = W - \text{item weight}$

total Profit $+=$ item Profit

else:

Aim:

Given n items, each with a given weight and value, you need to determine the maximum total value that can be obtained by placing these items into a knapsack with a capacity of W .

Input Format:

- The first line contains two space-separated integers: n - the number of items, W - the maximum capacity of the knapsack.
- The second line contains $2 \times n$ space-separated integers: Each pair of integers represents the value and weight of an item, respectively.

Output Format:

- The output prints a single floating-point number - the maximum total value that can be obtained in the knapsack. The value must be formatted to exactly six decimal places.

Example 1:

Input:

```
3 50 60 10 100 20 120 30
```

Output:

```
240.000000
```

Explanation:

- Take the item with a value of 60 and a weight of 10, along with the item valued at 100 with a weight of 20.
- Since the remaining knapsack capacity is 20, a portion of the third item, which has a value of 120 and a weight of 30, is included.
- The fraction taken is $120/30$, resulting in a value of $(120/30) * 20 = 80$.
- Thus, the total maximum value that can be obtained is: $60 + 100 + 80 = 240.000000$.
- This is the optimal value achievable within the given knapsack capacity.

Example 2:

Input:

```
2 50 60 10 100 20
```

Output:

```
160.000000
```

Explanation:

- Take both items completely, without breaking.
- The total maximum value of the item we can have is 160.000000 from the given capacity of a sack.

Constraints:

- $1 \leq n \leq 10^5$
- $1 \leq w \leq 10^9$
- $1 \leq \text{value}_i, \text{weight}_i \leq 10^4$

Note:

- Utilize the Fractional Knapsack approach, where you can take fractional parts of each item. This allows you to split items and include only a portion of them in the knapsack to achieve the highest possible total value.

Source Code:

CTC31099.c

```
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    int value;
    int weight;
    double ratio;
} Item;

// Comparator for sorting by ratio in descending order
int compare(const void *a, const void *b) {
    Item *i1 = (Item *)a;
    Item *i2 = (Item *)b;

    if (i1->ratio < i2->ratio) return 1;
    if (i1->ratio > i2->ratio) return -1;
    return 0;
}

int main() {
```



```

int n, capacity;
scanf("%d %d", &n, &capacity);

Item items[n];

// Read values and weights
for (int i = 0; i < n; i++) {
    scanf("%d %d", &items[i].value, &items[i].weight);
    items[i].ratio = (double)items[i].value / items[i].weight;
}

// Sort items by value/weight ratio
qsort(items, n, sizeof(Item), compare);

double totalValue = 0.0;
int remainingCapacity = capacity;

for (int i = 0; i < n && remainingCapacity > 0; i++) {
    if (items[i].weight <= remainingCapacity) {
        // Take full item
        totalValue += items[i].value;
        remainingCapacity -= items[i].weight;
    } else {
        // Take fractional part
        totalValue += items[i].ratio * remainingCapacity;
        remainingCapacity = 0;
    }
}

// Output result with exactly 6 decimal places
printf("%.6f\n", totalValue);

return 0;
}

```

Execution Results - All test cases have succeeded!

User Output

3 50

60 10 100 20 120 30

240.000000

Test Case - 1

User Output

2 50

60 10 100 20

160.000000

Test Case - 2

take fraction = $w / \text{item weight}$
 total Profit + = item Profit $\times$ fraction
 break

O/P $\rightarrow$ total Profit.

Example

Item	Profit	weight	Profit / weight
A	60	10	6
B	100	20	5
C	120	30	4

knapack capacity = 50

Selection:

- Take A $\rightarrow$ weight 10 $\rightarrow$ profit 60
- Take B $\rightarrow$ weight 20 $\rightarrow$ profit 100
- Remaining capacity = 20
- Take 20/30 of C $\rightarrow$ profit = $120 \times (20/30) = 80$

Total profit $\rightarrow 60 + 100 + 80 = 240$.

Conclusion - Hence, we studied fractional knapsack problem using greedy method.

$$\frac{3}{3} + \frac{7}{7} + \frac{4}{5} = \frac{14}{5}$$

Aim: Dijkstra shortest path algorithm (Handson path optimization) + time complexity analysis.

Theory: Dijkstra algorithm is used to find the shortest path from one starting point to all other points in a graph.

- Graph means places (nodes) connected by roads (edges).
- Each road has a distance or cost.
- The algorithm always choose the nearest unvisited node first.

ARISE & SHINE

Because it always selects the shortest option, it is called a greedy algorithm.

Why is it called path optimization?

Path optimization means:

- Finding the best path
- Using minimum distance
- Saving time, cost and resources.

This is why Dijkstra algorithm is widely used in:

- GPS navigation
- network routing
- traffic management

Algorithm (Handson path optimization steps).

Step 1: Initialization

- set the distance of the source node = 0
- set distance of all other nodes = infinity (∞)
- mark all nodes as unvisited.

Step 2: select Nearest node

- Choose the unvisited node with the smallest distance.
- Mark it as visited.

ARISE & SHINE

Step 3: update neighbor distances

- For each neighboring node.

New distance = current node

distance + edge weight

- If new distance is smaller, update it.

Step 4: Repeat

- Repeat step 2 and 3
- continue until all nodes are visited

Aim:

Write a program to implement Dijkstra's Algorithm to find the shortest path from a given source vertex to all other vertices in an undirected weighted graph.

Input Format:

The first line contains an integer V representing the number of vertices.

The second line contains an integer E representing the number of edges.

The next E lines contain three space-separated integers S , D , and W , where:

- S represents the starting vertex of the edge.
- D represents the destination vertex of the edge.
- W represents the weight of the edge.

The last line contains an integer S , which is the source vertex from where the shortest paths are to be calculated.

Output Format:

Print V lines, each containing two space-separated integers A and B :

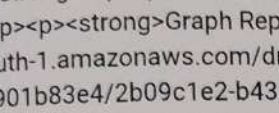
- A --> The vertex number.
- B --> The shortest distance from the source vertex to that vertex.

Example:

Input:

```
5 6 0 1 2 0 2 4 1 2 1 1 3 7 2 4 3 3 4 1 0
```

Graph Representation:



Output:

```
0 0 1 2 2 3 3 9 4 6
```

The shortest distances from source vertex 0 to all other vertices are:

- 0 → 0: Distance = 0
- 0 → 1: Distance = 2
- 0 → 2: Distance = 3 (0 → 1 → 2)
- 0 → 3: Distance = 9 (0 → 1 → 2 → 4 → 3)
- 0 → 4: Distance = 6 (0 → 1 → 2 → 4)

Constraints:

- $1 \leq V \leq 100$
- $0 \leq E \leq \frac{V(V-1)}{2}$
- $0 \leq S, D \leq V-1$
- $1 \leq W \leq 100$

The graph is undirected and connected.

Source Code:

CTC64797.c

```
#include <stdio.h>
#include <limits.h>
#include <stdbool.h>

#define MAX 1000

void dijkstra(int n, int graph[MAX][MAX], int src) {
    int dist[MAX];
    bool visited[MAX];

    for (int i = 0; i < n; i++) {
        dist[i] = INT_MAX;
        visited[i] = false;
    }

    dist[src] = 0;

    for (int count = 0; count < n - 1; count++) {

        int min = INT_MAX, u;

        for (int v = 0; v < n; v++) {
            if (!visited[v] && dist[v] <= min) {
                min = dist[v];
                u = v;
            }
        }
    }
}
```

```

    }

    visited[u] = true;

    for (int v = 0; v < n; v++) {
        if (!visited[v] && graph[u][v] && dist[u] != INT_MAX
            && dist[u] + graph[u][v] < dist[v]) {
            dist[v] = dist[u] + graph[u][v];
        }
    }

    for (int i = 0; i < n; i++) {
        printf("%d %d\n", i, dist[i]);
    }
}

int main() {
    int n, e;
    int graph[MAX][MAX] = {0};

    if (scanf("%d %d", &n, &e) != 2) return 0;

    for (int i = 0; i < e; i++) {
        int u, v, w;
        scanf("%d %d %d", &u, &v, &w);
        graph[u][v] = w;
        graph[v][u] = w;
    }

    int src;
    scanf("%d", &src);

    dijkstra(n, graph, src);

    return 0;
}

```

Execution Results - All test cases have succeeded!

User Output	Test Case - 1
5	
6	
0 1 2	
0 2 4	
1 2 1	
1 3 7	
2 4 3	
3 4 1	
0	
0 0	

1 2
2 3
3 7
4 6

User C
4
4
0 1 5
0 2 1
1 2 2
1 3 3
0
0 0
1 5
2 7
3 8

u] != INT_MAX

1 2
2 3
3 7
4 6

User Output

Test Case - 2

4

4

0 1 5

0 2 10

1 2 2

1 3 3

0

0 0

1 5

2 7

3 8

Step 5: Result

- The final distances represent the shortest path from the source.

Time Complexity Analysis

Time complexity tells us how fast or slow the algorithm works.

Case 1: using simple Array

- Each node is checked again and again
- Time complexity: $O(V^2)$
- V = number of nodes.

Case 2: using priority queue (heap)

- Faster way to select nearest node.
- Time Complexity: $O((V+E) \log V)$
- V = vertices
- E = edges.

ARISE & SHINE

Conclusion: Hence, we studied Dijkstra shortest path algorithm (Handwritten path optimization) + time complexity analysis.

$$\frac{3}{3} + \frac{7}{7} + \frac{4}{5} = \frac{14}{5}$$

Aim: Minimum cost spanning tree using Kruskal's algorithm (union find + optimization).

Theory: A minimum spanning tree (MST) is:

- A tree formed from a connected, undirected, weighted graph.
- connects all vertices
- Has no cycles
- Has minimum total edge cost.

If a graph has V vertices, the MST will have exactly $V-1$ edges.

Kruskal's Algorithm - Kruskal's Algorithm is a greedy algorithm used to find the MST

- It selects the smallest weight edge first
- It avoids forming cycles.
- It continues until we get $V-1$ edges.

Main idea of Kruskal's Algorithm.

- (1) sort all edges in ascending order of weight.
- (2) Pick the smallest edge.
- (3) Check if adding it creates a cycle.
- (4) If no cycle $\rightarrow$ add it.

(5) If cycle $\rightarrow$ discard it

(6) Repeat until MST is formed.

Why do we use union - find?

To detect cycles efficiently, we use a data structure called:

Disjoint set (Union - find)

• Find $() \rightarrow$ find which set a vertex belongs to

• Union $() \rightarrow$ merge two sets.

union - find optimization -

1. Path compression - When performing find $()$, make nodes directly point to root.
makes future operations faster.

2. Union by Rank (or size)

Attach smaller tree under larger tree.
keeps tree height small.

How cycle detection works.

If two vertices belongs to the same set, adding that edge will form a cycle.

If they belongs to different sets, we can safely add the edge.

Step-by-step example.

edges $\rightarrow$	weight	Step 1 $\rightarrow$ sort edges
A-B	1	1 $\rightarrow$ A-B
B-C	2	2 $\rightarrow$ B-C
A-C	3	3 $\rightarrow$ A-C
C-D	4	4 $\rightarrow$ C-D

Step 2 $\rightarrow$ Kruskal's Picking

- pick A-B
- pick B-C
- pick A-C (cycle)
- pick C-D

mst formed.

Time complexity

Step complexity

Sorting edges $\rightarrow O(E \log E)$

Union-find operation $\rightarrow$ Almost $O(1)$

Total $- O(E \log E)$

Very efficient for large graphs.

Aim:

To find the minimum cost spanning tree of a given undirected graph using Kruskal's algorithm.

Input Format:

- The first line of input reads a positive integer n , representing the number of vertices.
- The next n lines contain n space-separated integers representing the cost adjacency matrix of the graph, with a value of 0 indicating no edge between the vertices.

Output Format:

- A single integer representing the minimum cost of the spanning tree.

Source Code:

CTC66970.c

```
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    int u, v, weight;
} Edge;

int find(int parent[], int i) {
    if (parent[i] == i)
        return i;
    return parent[i] = find(parent, parent[i]);
}

void unite(int parent[], int rank[], int x, int y) {
    int rootX = find(parent, x);
    int rootY = find(parent, y);
    if (rootX != rootY) {
        if (rank[rootX] < rank[rootY])
            parent[rootX] = rootY;
        else if (rank[rootX] > rank[rootY])
            parent[rootY] = rootX;
        else {
            parent[rootY] = rootX;
            rank[rootX]++;
        }
    }
}

int compareEdges(const void* a, const void* b) {
    return ((Edge*)a)->weight - ((Edge*)b)->weight;
}

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

    Edge edges[n * n];
```



```

int edgeCount = 0;

for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        int weight;
        scanf("%d", &weight);

        if (weight > 0 && i < j) {
            edges[edgeCount].u = i;
            edges[edgeCount].v = j;
            edges[edgeCount].weight = weight;
            edgeCount++;
        }
    }
}

qsort(edges, edgeCount, sizeof(Edge), compareEdges);

int parent[n], rank[n];
for (int i = 0; i < n; i++) {
    parent[i] = i;
    rank[i] = 0;
}

int minCost = 0;
int edgesIncluded = 0;

for (int i = 0; i < edgeCount && edgesIncluded < n - 1; i++) {
    int rootU = find(parent, edges[i].u);
    int rootV = find(parent, edges[i].v);

    if (rootU != rootV) {
        minCost += edges[i].weight;
        unite(parent, rank, rootU, rootV);
        edgesIncluded++;
    }
}

printf("%d\n", minCost);

return 0;
}

```

Execution Results - All test cases have succeeded!

Test Case - 1	
User Output	
3	
0 2 3	
2 0 1	
3 1 0	
3	

Test Case - 2

User Output

4
0143
1025
4206
3560

why Kruskal is good.

- works well for sparse graph.
- easy to implement
- efficient with union-find.

Application.

- Network design (LAN cables)
- Road construction
- electrical wiring
- clustering (machine learning)

conclusion: Hence we studied, minimum cost spanning tree using ~~Kruskal's~~ algorithm (union find + optimization).

ARISE & SHINE

$$\frac{3}{3} + \frac{7}{7} + \frac{4}{5} = \frac{14}{5}$$

MSD

27-2-26

Aim: Minimum cost spanning tree using Prim's algorithm (Adjacency matrix) and its adjacency list performance comparison.

Theory: A minimum spanning tree (MST)

- A tree made from a graph.
- Connects all vertices (nodes)
- Has no cycle (no loops)
- Has minimum total edge weight (cost)
- Used in network design (roads, cables, water pipes).

Prim's algorithm is a greedy algorithm used to find minimum spanning tree (MST) of a weighted, connected graph.

It starts from any one vertex and keeps adding the smallest edge that connects a new vertex to the tree.

Algorithm of Prim's

1. Start from any one node.

2. Look at all edges connected to it.
3. Pick the edge with smallest weight.
4. Add the vertex to your tree.
5. Repeat until all vertices are included.

ex: Consider the graph $\Rightarrow$ vertices: A, B, C, D
 edges $\rightarrow$ $\cdot A-B=2$ $\cdot A-C=3$ $\cdot B-C=1$
 $\cdot B-D=4$ $\cdot C-D=5$

Apply prim's Algorithm.

Start from A

Step 1: edges from A $\rightarrow (A-B=2), (A-C=3)$

Choose smallest $\rightarrow A-B(2)$

Step 2: New visited: A, B

Edges available: $B-C=1, B-D=4, A-C=3$

Choose smallest $\rightarrow B-C(1)$

Step 3: New visited: A, B, C

Edges available: $B-D=4, C-D=5$

Choose smallest $\rightarrow B-D(4)$

Now all vertices are connected.

Final mst edges:

$\cdot A-B=2$

$\cdot B-C=1$

$\cdot B-D=4$

total cost = $2+1+4=7$

Aim:

Write a C program to find the Minimum Spanning Tree (MST) of a connected, weighted, undirected graph using Prim's Algorithm.

You are given a partially implemented driver code. Your task is to complete the missing logic in the `minKey()`, `prim()` functions. The program represents the graph using an adjacency matrix.

Functions to be Completed

1. `minKey()`: This function selects the vertex that has the minimum key value from the set of vertices not yet included in the MST.

```
int minKey(int key[], bool mstSet[], int vertices)
```

- `key[]` stores the minimum weight edge connecting a vertex to the MST
- `mstSet[]` keeps track of vertices already included in the MST
- The function returns the index of the vertex with the smallest key value that is not yet in the MST

2. `prim()`: This function implements Prim's Algorithm to construct the Minimum Spanning Tree.

```
void prim(int graph[V][V], int vertices)
```

- This function finds the MST of a graph by starting from one vertex and repeatedly adding the nearest (lowest weight) vertex until all vertices are connected.

Input Format:

- The program first prompts the user to enter an integer vertices represents the number of vertices in the following format:

```
No of vertices:
```

- Next line, the program prompts:

```
Adjacency matrix elements (row wise):
```

- The next vertices lines each contain vertices space-separated integers representing the adjacency matrix.
- The value at row `i` and column `j` denotes the weight of the edge between vertex `i` and vertex `j`.
- A value of 0 indicates that there is no edge between the corresponding vertices.

Output Format:

- After constructing the Minimum Spanning Tree, the program prints the header:

```
Edge Weight
```

- Then, for each edge selected in the MST, print:

```
<u> - <v> <weight>
```

Where:

- `<u>` is the parent vertex of `v` in the MST
- `<v>` is the current vertex

Constraints:

- $1 \leq \text{vertices} \leq 100$
- Edge weights are non-negative integers
- The graph is connected
- The adjacency matrix is symmetric (undirected graph)

Note:

- The MST always starts from vertex 0
- Refer to the visible test cases for better understanding.

Source Code:

minCostFinding.c

```
#include <stdio.h>
#include <stdbool.h>
#include <limits.h>
#define V 100

int minKey(int key[], bool mstSet[], int vertices) {
    // Write your code here...
    int min = INT_MAX;
    int min_index = -1;

    for(int v=0;v<vertices;v++)
    {
        if(!mstSet[v]){

            if(key[v] < min || (key[v]==min && v < min_index)){

                min = key[v];
                min_index=v;
            }
        }
    }
    return min_index;
}

void printTree(int parent[], int graph[V][V], int vertices) {
    printf("Edge \tWeight\n");
    for (int i = 1; i < vertices; i++)
        printf("%d - %d \t%d \n", parent[i], i, graph[i][parent[i]]);
}

void prim(int graph[V][V], int vertices) {
    // Write your code here...
    int parent[V];
    int key[V];
    bool mstSet[V];
```

```

for(int i=0;i<vertices;i++){
    key[i]=INT_MAX;
    mstSet[i]=false;
}
key[0]=0;
parent[0]=-1;

for(int count =0;count <vertices -1;count++){
    int u =minKey(key,mstSet,vertices);
    mstSet[u]=true;

    for(int v=0;v<vertices;v++){
        if(graph[u][v] !=0 && !mstSet[v] && graph[u][v] < key[v]){
            parent[v]=u;
            key[v] = graph [u][v];
        }
    }

    printf("Edge \tWeight\n");
    for(int i=1;i<vertices;i++){
        printf("%d - %d \t%d \n",parent[i],i,graph[i][parent[i]]);
    }
}

}

int main() {
    int vertices;
    int graph[V][V];

    printf("No of vertices: ");
    scanf("%d", &vertices);

    printf("Adjacency matrix elements (row wise):\n");
    for (int i = 0; i < vertices; i++) {
        for (int j = 0; j < vertices; j++) {
            scanf("%d", &graph[i][j]);
        }
    }

    prim(graph, vertices);

    return 0;
}

```

Execution Results - All test cases have succeeded!

Test Case - 1

No of vertices:

5

Adjacency matrix elements (row wise):

0 0 4 0 0

0 0 5 3 0

4 5 0 0 0

0 3 0 0 2

0 0 0 2 0

Edge	Weight
------	--------

2 - 1	5
-------	---

0 - 2	4
-------	---

1 - 3	3
-------	---

3 - 4	2
-------	---

time complexity

Implementation

simple (matrix)

using min heap +

Adjacency list

time complexity

$$O(V^2)$$

$$O(E \log V)$$

where :

• V = number of vertices

• E = number of edges.

prim's greedy - Because at every step it chooses the smallest available edge without thinking about future.

prim's algorithm is a greedy algorithm used to find the minimum spanning tree of a connected weighted graph.

It starts from any vertex and repeatedly adds the ~~smallest edge~~ that connects a new vertex ~~to the tree~~.

Conclusion - Hence, we studied minimum cost spanning tree using prim's algorithm (Adjacency matrix vs adjacency list performance comparison).

$$\frac{3}{8} + \frac{7}{7} + \frac{3}{5} = \frac{13}{15}$$

Aim: All pairs shortest path using Floyd-Warshall algorithm (time complexity $O(n^3)$ demonstration).

Theory: The Floyd-Warshall Algorithm is used to find the shortest path between all pairs of vertices in a weighted graph.

- "All Pairs Shortest Path" means we find the shortest distances from every vertex to every other vertex.
- It works for directed and undirected graphs.
- It can also work when edge weights are negative, but ~~negative cycles~~ should not exist.

The algorithm works by checking every vertex as an intermediate point between two vertices.

Suppose we want the shortest path from $A \rightarrow C$

• $A \rightarrow B \rightarrow C$

• $A \rightarrow D \rightarrow C$

• $A \rightarrow E \rightarrow C$

If going through another vertex gives shorter distance, the algorithm updates the distance.

The process repeats for all vertices.

Algorithm

1. Create a distance matrix from the graph.

2. If there is no edge, put ∞ (infinity)

3. Take each vertex k as an intermediate vertex.

4. For every pair (i, j) :

• Check if $i \rightarrow k \rightarrow j$ is shorter than $i \rightarrow j$.

5. If yes, update the distance.

6. Repeat for all vertices.

Example: Initial distance matrix:

	A	B	C
A	0	5	∞
B	∞	0	3
C	2	∞	0

Aim:

You are given a weighted directed graph with N vertices and E edges. Your task is to implement the Floyd-Warshall algorithm to compute the shortest paths between all pairs of vertices in the graph. Vertices are numbered from 1 to N .

Requirements:

- Use the Floyd-Warshall algorithm to compute all-pairs shortest paths.
- Initialize the adjacency matrix where 0 is on the diagonal, given edge weights, and INF for all other entries.
- After computing shortest paths, output the final matrix.

Input Format:

- The first line contains an integer N , representing the number of vertices in the graph.
- The second line contains an integer E , representing the number of edges in the graph.
- The next E lines each contain three space-separated integers s , d , and w , where s is the source vertex, d is the destination vertex, and w is the weight of the edge from s to d .

Output Format:

Print the $N \times N$ shortest path matrix, where:

- Entry at row i , column j represents the shortest distance from the vertex i to the vertex j , with a trailing space after the last element on each row.
- If no path exists between i and j , print "INF".

Example:

Input

```
4
5
1 2 4
1 4 10
1 3 6
2 4 5
3 4 2
```

Output

```
0 4 6 8
INF 0 INF 5
INF INF 0 2
INF INF INF 0
```

Explanation:

- The matrix shows the shortest distances between all pairs of vertices.
- INF denotes that there is no path between the respective vertices.

Constraints:

- $1 \leq N \leq 20$
- $1 \leq E \leq N \times (N - 1)$
- $1 \leq w \leq 99999$

Source Code:

CC83047.c

```
#include<stdio.h>
```

```
#define MAX 20
```

```
#define INF 1000000000
```

```
int main(){
```

```
    int n,m;
```

```
    int dist[MAX][MAX];
```

```
    scanf("%d",&n);
```

```
    scanf("%d",&m);
```

```
    for(int i=0;i<n;i++){
```

```
        for(int j=0;j<n;j++){
```

```
            if(i==j)
```

```
                dist[i][j]=0;
```

```
            else
```

```
                dist[i][j]=INF;
```

```
        }
```

```
    }
```

```
    for(int i=0;i<m;i++){
```

```
        int u,v,w;
```

```
        scanf("%d %d %d", &u,&v,&w);
```

```
        dist[u-1][v-1] =w;
```

```
    }
```

```
    for(int k=0;k<n;k++){
```

```
        for(int i=0;i<n;i++){
```

```
            for(int j =0;j<n;j++){
```

```
                if(dist[i][k] + dist[k][j] < dist[i][j]){
```

```
                    dist[i][j] = dist[i][k]+dist[k][j];
```

```
            }
```

```
        }
```

```
    }
```

```
}
```

```
for(int i=0;i<n;i++){
```

```
    for(int j=0;j<n;j++){
```

```
        if(dist[i][j]==INF)
```

```
            printf("INF ");
```

```
        else
```

```
            printf("%d ",dist[i][j]);
```

```
    }
```

```
    printf("\n");
```

```
}
```

```
return 0;
```


Execution Results - All test cases have succeeded!

Test Case - 1

User Output

4

5

1 2 4

1 4 10

1 3 6

2 4 5

3 4 2

0 4 6 8

INF 0 INF 5

INF INF 0 2

INF INF INF 0

Test Case - 2

User Output

4

5

1 2 4

3 2 5

4 1 1

4 2 3

4 3 8

0 4 INF INF

INF 0 INF INF

INF 5 0 INF

1 3 8 0

Now check intermediate vertices.

ex: $A \rightarrow B \rightarrow C = 5 + 3 = 8$

Since, $8 < \infty$ update:

$A \rightarrow C = 8$

final shortest paths will be calculated after checking all vertices.

Time Complexity.

The algorithm uses three nested loops:

- loop for k
- loop for i
- loop for j

so total operations: $n \times n \times n = O(n^3)$

Advantage: Finds shortest path between all pairs of vertices.

Disadvantage: High time complexity $O(n^3)$ for large graphs.

Conclusion: Hence we studied, all pairs shortest path using Floyd warshall algorithm.
(Time complexity $O(n^3)$ demonstration.)

Aim: Travelling salesman problem (TSP) using dynamic programming (small inputs sets + exponential complexity observation).

Theory: A salesman must visit all cities exactly once and return to the starting city while minimizing the total travel cost or distance.

Dynamic programming helps reduce repeated calculations.

Algorithm: 1. Start from city 0 (A).

2. Visit cities one by one.

3. Store minimum cost in dynamic program table.

4. After visiting all cities, return to starting city.

5. Choose minimum total cost.

Time complexity: $O(n^2 \times 2^n)$ → for each state we check previous city option.
 n = Number of cities.

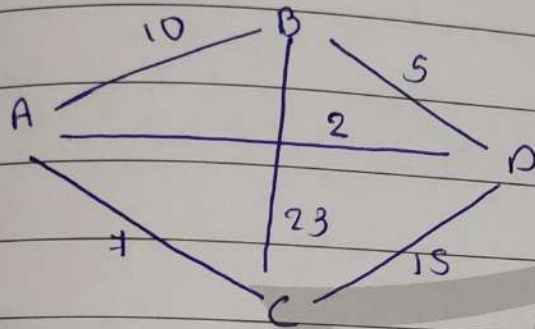
Space complexity: DP table size → $O(n \times 2^n)$ because we store result for each city and subset.

Dynamic Programming for TSP

Advantages:

1. Reduces repeated calculations.
2. Saves time & brute force.
3. Gives exact minimum path.

Ex:-



Possible paths = $(V-1)! = 6$

$$A-B-C-D-A = 50$$

$$A-C-B-D-A = 37$$

$$A-D-B-C-A = 37$$

$$A-D-C-B-A = 50$$

$$A-B-D-C-A = 37$$

$$A-C-D-B-A = 37$$

Algorithm - (i) Start

1. Create a possible path of the given graph.
2. After visiting all the vertices return to the starting point.
3. Choose minimum total cost.
4. End.

Aim:

You are given a number of cities and a cost matrix that represents the cost of traveling between every pair of cities. The goal is to find the minimum cost of a tour that:

- Starts from city 0,
- Visits each city exactly once, and
- Returns to the starting city.

Use Dynamic Programming with Bitmasking to implement a solution for the Travelling Salesman Problem (TSP).

Input Format:

- The first line contains a positive integer n , representing the number of cities.
- The next n lines each contain n space-separated integers, representing the adjacency matrix of the graph.

Output Format:

- Prints a single integer, which is the minimum travel cost.

Source Code:

CTC73914.c

```
#include <stdio.h>
#include <limits.h>
#include <string.h>
#define MAXN 20

int n;
long long dist[MAXN][MAXN];
long long dp[1 << MAXN][MAXN];

long long min(long long a, long long b) {
    return (a < b) ? a : b;
}

long long tsp(int mask, int pos) {
    if (mask == (1 << n) - 1) {
        return dist[pos][0];
    }

    if (dp[mask][pos] != -1)
        return dp[mask][pos];

    long long ans = LLONG_MAX;
    for (int city = 0; city < n; city++) {
        if ((mask & (1 << city)) == 0) {
            long long newAns = dist[pos][city] +
tsp(mask | (1 << city), city);
            ans = min(ans, newAns);
        }
    }
}
```

```

    }
    return dp[mask][pos] = ans;
}

int main() {
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%lld", &dist[i][j]);
        }
    }

    memset(dp, -1, sizeof(dp));

    long long result = tsp(1, 0);
    printf("%lld ", result);

    return 0;
}

```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
4
0 10 15 20
10 0 35 25
15 35 0 30
20 25 30 0
80

Test Case - 2
User Output
3
0 20 30
20 0 10
30 10 0
60

Applications of TSP:-

TSP is used in real life:

1. Delivery route optimization.
2. Courier services.
3. Airline route planning.
4. Robot Path Planning.
5. Circuit board manufacturing.

Travelling salesman problem (TSP) is the problem of finding the shortest route that visits every city exactly once and returns to the starting city. Dynamic programming solves it by breaking solutions of smaller subproblems, with time complexity $O(n^2 \times 2^n)$.

Conclusion = Hence we studied, travelling salesman problem (TSP) using dynamic programming (small inputs sets + exponential complexity observations.)

$$\frac{3}{3} + \frac{7}{7} + \frac{4}{5} = \frac{14}{5}$$

(MSR)

25-3-26

Aim: 0/1 knapsack problem using dynamic programming (bottom up dynamic programming) compare results with greedy fractional knapsack.

Theory: you have a bag (knapsack) with limited capacity (weight limit) you have items with:

- weight
- value

Take items so that total value is maximum but weight $\leq$ capacity.

types of knapsack -

(A) 0/1 knapsack

- you can take full item or not take it.
- No breaking allowed.

(B) Fractional knapsack

- you can take full or part of item.
- Breaking allowed.

Item	weight	value
1	1	10
2	3	40
3	4	50
4	5	70

Capacity = 8

Algorithm : 1] create table

2] Fill with 0

3] Loop through items.

4] for each weight :

- if item fits $\rightarrow$ choose max
- else $\rightarrow$ skip.

Ans + Item 2 (40)
Item 4 (70)

Total weight = $3 + 5 = 8$

Total value = 110.

Fractional knapsack (greedy)

Item	value	weight	Ratio
		1	10
1	10		13.3
2	40	3	12.5
3	50	4	14
4	70	5	

Aim:

You are given n items, each with a weight and a value. Your task is to determine the maximum total value that can be obtained by placing items into a knapsack of capacity W . Each item can either be taken completely or left (0/1 Knapsack). Use Dynamic Programming to solve the problem.

Input Format:

- The first line contains two integers n and W representing the number of items and the knapsack capacity.
- Each of the next n lines contains two integers: $weight_i$ and $value_i$.

Output Format:

- Print a single integer, the maximum total value achievable.

Sample Input:

```
4 5
2 3
1 2
3 4
2 2
```

Sample Output:

```
7
```

Explanation:

1. We are given 4 items with weights [2, 1, 3, 2] and values [3, 2, 4, 2].
2. The knapsack has capacity $W = 5$.
3. The goal is to maximize the total value without exceeding the weight limit.

a. Case 1: Choose only 1 item

- Item 3 has a weight of 3, which is less than or equal to 5, and it provides the maximum value of 4, so it is selected.

b. Case 2: Choose 2 items

- Items (1,2): weight = $2+1 = 3 \rightarrow$ value = $3+2 = 5$
- Items (1,3): weight = $2+3 = 5 \rightarrow$ value = $3+4 = 7$ (maximum so far)
- The same process is applied for other pairs, but none exceeds 7.
- So, the maximum with 2 items = 7.

4. For the remaining cases (choosing 3 items or all 4 items), the same process is applied:

- Maximum value with 3 items = 7 (Items 1,2,4).
- All 4 items cannot be selected since the total weight > 5 .

5. Among all possibilities, the maximum achievable value is 7, which can be obtained either by selecting Items (1,3) or Items (1,2,4).

6. Thus, the maximum value of the knapsack is: 7.

Constraints:

- $1 \leq n \leq 1000$
- $1 \leq W \leq 10000$
- $1 \leq weight_i, value_i \leq 10000$

Source Code:


```
#include <stdio.h>
```

```
int max(int a, int b) {  
    return (a > b) ? a : b;  
}
```

```
int main() {  
    int n, W;
```

```
    // Input number of items and capacity  
    scanf("%d %d", &n, &W);
```

```
    int weight[n], value[n];
```

```
    // Input weights and values  
    for (int i = 0; i < n; i++) {  
        scanf("%d %d", &weight[i], &value[i]);  
    }
```

```
    // DP table  
    int dp[n + 1][W + 1];
```

```
    // Initialize DP table  
    for (int i = 0; i <= n; i++) {  
        for (int w = 0; w <= W; w++) {  
            if (i == 0 || w == 0)  
                dp[i][w] = 0;  
            else if (weight[i - 1] <= w)  
                dp[i][w] = max(value[i - 1] + dp[i - 1][w - weight[i - 1]],  
                                dp[i - 1][w]);  
            else  
                dp[i][w] = dp[i - 1][w];  
        }  
    }
```

```
    // Output maximum value  
    printf("%d", dp[n][W]);
```

```
    return 0;
```

```
}
```

Execution Results - All test cases have succeeded!

Test Case - 1	
User Output	
4 5	
2 3	
1 2	
3 4	
2 2	

Test Case - 2

User Output

57

11

34

45

57

23

10

Step 2: sort by ratio (descending)

Item 4 $\rightarrow$ Item 2 $\rightarrow$ Item 3 $\rightarrow$ Item 1

Step 3: Fill bag

• Take item 4 $\rightarrow$ weight 5 $\rightarrow$ value 70

• Remaining capacity = 3

• Take item 2 $\rightarrow$ weight 3 $\rightarrow$ value 20

• Total = 110.

Use 0/1 knapsack (DP) when:

- Items cannot be broken
- Need exact answer.

Use fractional (greedy) when:

- Items can be divided
- Need faster solution.

ARISE & SHINE

Conclusion: Hence, we studied 0/1 knapsack problem using dynamic programming (bottom up dynamic programming) compare results with greedy fractional knapsack.

$$\frac{3}{3} + \frac{7}{7} + \frac{3}{5} = \frac{13}{5}$$

MRD E
10-11-26

Aim : 8 Queens problem using backtracking (mode explanation + training technique)

Theory : You have a chessboard (8×8) and 8 queens.

We must place all 8 queens such that :

- No two queens attack each other.

Queens attack :

- same row
- same column
- same diagonal

What is backtracking.

Backtracking means : Try $\rightarrow$ if wrong $\rightarrow$ Go back $\rightarrow$ Try another option.

Each position is like a node in a tree

Example (for 4 queens for simplicity) :

Row 0 $\rightarrow$ try col 0, 1, 2, 3
1

Row 1 $\rightarrow$ try all columns again
1

Row 2 $\rightarrow$ Continue ...

- each choice = a node
- exploring all choices = node exploration.

Algorithm :

- (1) start from row 0
- (2) for each column :
check if safe
if safe :
place queen
move to next row

- (3) if all queen placed :
print solution

else :

Back track (remove queen)

if not safe :

try next column.

ARISE & SHINE

time complexity

worst case $\rightarrow O(N!)$

because we try many possibilities.

key points

- (1) place queen row by row
- (2) check safety before placing
- (3) if stuck $\rightarrow$ back track
- (4) explore all modes (no. explanation)

Aim:

To Implement N Queen's problem using Backtracking. Place n queens on an $n \times n$ chessboard such that no two queens attack each other.

Input Format:

- The first line contains a single integer n , representing the number of queens.

Output Format:

- For each valid solution, print "Solution <number>:" followed by the $n \times n$ chessboard representation.
- Each row contains n space-separated elements: "Q" represents a queen, and "*" represents an empty position, with a trailing space after the last element on each row.
- After printing all solutions, print "Total solutions = <count>" on a new line, where <count> is the total number of valid solutions found.
- If no solutions found then print "Total solutions = 0".

Constraints:

- $1 \leq n \leq 10$

Note:

- Refer to the visible test cases to match the input/output format strictly.

Source Code:

CTC66973.c

```
#include<stdio.h>

int n,c = 0,col[20],d1[40],d2[40],pos[20];

void solve(int r){
    if(r==n){
        printf("Solution %d:\n", ++c);
        for(int i=0;i<n;i++){
            for(int j=0;j<n;j++){
                printf(j==pos[i] ? "Q " : "* ");
            }
            printf("\n");
        }
        return;
    }

    for(int j=0;j<n;j++){
        if(!col[j] && !d1[r-j+n] && !d2[r+j]){
            pos[r] = j;
            col[j] = d1[r-j+n] = d2[r+j] = 1;
            solve(r+1);
            col[j] = d1[r-j+n] = d2[r+j] = 0;
        }
    }
}

int main(){
```



```
solve(c);  
printf("Total solutions = %d\n",c);  
return 0;
```

Execution Results - All test cases have succeeded!

Test Case - 1

User Output

4

Solution 1:

* Q * *

* * * Q

Q * * *

* * Q *

Solution 2:

* * Q *

Q * * *

* * * Q

* Q * *

Total solutions = 2

Test Case - 2

User Output

3

Total solutions = 0

The 8 queens problem is a classic problem in backtracking.

It belongs to :

- Artificial Intelligence
- Algorithm Design
- Constraint Satisfaction problems (CSP).

state space tree.

The solution can be represented using a state space tree.

What is it?

- Each node = partial solution.
- Each level = row of chessboard
- Each branch = column choice.

Space complexity

(1) Recursive stack : $O(n)$

(2) Board : $O(n^2)$.

Application : time consuming. N queens problem, Sudoku puzzle, maze solving, Graph coloring.

Conclusion : Hence, we solved 8 queens problem using backtracking.

$$\frac{3}{2} + \frac{7}{7} + \frac{4}{5} = \frac{14}{15}$$

Aim: Graph colouring problem (chromatic number computation of a sample graph).

Theory: What is backtracking $\rightarrow$ try a solution $\rightarrow$ if it fails $\rightarrow$ go back $\rightarrow$ try another option. It is like exploring all possible paths until you find the correct one.

Real life example:

You are trying to open a lock with 3 digits.

• try 123

• try 124

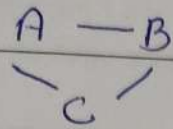
• try 125

You keep trying and go back when wrong.

Graph colouring: No two neighbouring regions touching each other should have the same color.

Chromatic number: The minimum number of colors needed to color a graph properly.

ex: Suppose we have 3 nodes forming a triangle.



connections :
• A connected to B and C.
• B connected to C.

try coloring :
A = red
B = blue
C = ?

C cannot be red (A is red)

C cannot be blue (B is blue)

So, we need 3rd color

Chromatic Number = 3

eg $\rightarrow A - B - C$

coloring :
A = Red
B = Blue

C = Red (allowed, because not directly connected to A).

Chromatic Number = 2

Algorithm : This method tries different colors and checks what works.

You are given an undirected graph with n vertices represented by its adjacency matrix. Your task is to determine all valid colorings of the graph using at most m different colors, such that no two adjacent vertices share the same color.

This is known as the M-Coloring Problem, and it must be solved using the backtracking technique.

A coloring is considered valid if:

- Each vertex is assigned a color from 1 to m .
- No two vertices connected by an edge share the same color.

Input Format:

1. The first line contains a single integer n - the number of vertices in the graph.
2. The second line contains a single integer m - the number of colors available.
3. The next n lines each contain n integers, representing the adjacency matrix G , where:
 - $G[i][j] = 1$ if there is an edge between vertex i and vertex j
 - $G[i][j] = 0$ otherwise

Output Format:

1. Print each valid coloring of the graph on a new line.
2. Each coloring consists of n space-separated integers, where the i^{th} number represents the color assigned to vertex i .

Source Code:

CTC68355.c

```
#include <stdio.h>
#include <stdlib.h>

int count = 0;

int isSafe(int v, int n, int graph[n][n], int color[], int c) {
    for (int i = 0; i < n; i++)
        if (graph[v][i] == 1 && color[i] == c)
            return 0;
    return 1;
}

void solve(int v, int n, int m, int graph[n][n], int color[]) {
    if (v == n) {
        for (int i = 0; i < n; i++)
            printf("%d ", color[i]);
        printf("\n");
        count++;
        return;
    }

    for (int c = 1; c <= m; c++) {
        if (isSafe(v, n, graph, color, c)) {
            color[v] = c;
            solve(v + 1, n, m, graph, color);
        }
    }
}
```

```

int main() {
    int n, m;
    scanf("%d", &n);
    scanf("%d", &m);

    int graph[n][n];
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            scanf("%d", &graph[i][j]);

    int color[n];
    for (int i = 0; i < n; i++)
        color[i] = 0;

    solve(0, n, m, graph, color);

    if (count == 0)
        printf("0");

    return 0;
}

```

Execution Results - All test cases have succeeded!

Test Case - 1	
User Output	
4	
3	
0 1 1 1	
1 0 1 0	
1 1 0 1	
1 0 1 0	
1 2 3 2	
1 3 2 3	
2 1 3 1	
2 3 1 3	
3 1 2 1	
3 2 1 2	

Test Case - 2	
User Output	
3	
3	
0 1 1	
1 0 1	
1 1 0	
1 2 3	

1 3 2

2 1 3

2 3 1

3 1 2

3 2 1

- Steps :
- (1) start with first vertex.
 - (2) Assign a color
 - (3) move to next vertex
 - (4) Assign a color that is :
 - Not used by its neighbors.
 - (5) If no color is possible :
 - Go back (backtrack)
 - Try different color
 - (6) Repeat until all vertices are colored.

Real life uses:

- Timetable scheduling
- map coloring
- Register allocation in compilers.
- Frequency assignment.

Ex + Graph $\rightarrow$ $\begin{matrix} A-B \\ | \\ C \end{matrix}$ ARISE & SHINE

• A = Red

B = Blue

C = Blue (not connected to B directly)

Chromatic Number = 2.

Conclusion : Hence, we studied graph coloring problem (Chromatic number of a sample graph).

$$\frac{3}{3} + \frac{7}{7} + \frac{3}{5} = \frac{13}{5}$$

MR 2

Aim: subset sum | n-queens | pseudo code
solver using backtracking (any 1) (show
complexity, expression + solution pruning).

Theory: N Queen problem: place N queens
on an $N \times N$ chessboard so that;

- No two queens attack each other
- No same row
- No same column
- No same diagonal.

Backtracking concept: "try $\rightarrow$ check $\rightarrow$ if wrong $\rightarrow$
try next"

complexity expression:

- Row 1 $\rightarrow$ N choices
- Row 2 $\rightarrow$ N choices
- Row 3 $\rightarrow$ N choices

total possibilities $\rightarrow N \times N \times N \dots = N^N$

ex: $N = 4 \Rightarrow 256$ possibilities

$N = 8 = 16, 777, 216$ possibilities.

Pruning → If position is unsafe → stop exploring that path immediately.

ex: - If queen attacks → don't go deeper.
• skip whole branch.

so instead of checking all n^n = we check only valid partial solution.

After pruning complexity

- worst case : $O(n!)$
- But practically much smaller due to pruning.

why pruning is important:

without pruning → you check all possibilities (very slow)

with pruning:

- you skip wrong path early
- save time + computation.

What is subset sum problem?

• A set of numbers

• A target sum.

Ex: set = {2, 4, 6, 8}

target = 10

⇒ {2, 8} {4, 6}

we have 2 choices:

1. Take it.

2. Do not take it

we try all combinations.

Complexity

Each element has 2 choices:

Total subsets = 2^n

time complexity = $O(2^n)$

Conclusion: Hence, we studied subset sum / n-queens / pseudo code solver using backtracking. Short complexity + solution proving.

$$\frac{3}{3} + \frac{7}{7} + \frac{4}{5} = \frac{14}{15}$$

MPD

Aim:

In the Subset Sum Problem, the goal is to find a subset of a given array such that the sum of the elements in the subset equals a specified target number. A backtracking approach explores all possibilities efficiently, avoiding unnecessary computations in most cases.

Write a program to find if there exists at least one subset whose sum equals the given target number.

Input Format:

- The first line contains an integer *sum*, the target sum to find.
- The second line contains an integer *n*, the number of elements in the array.
- The third line contains *n* space-separated positive integers representing the elements of the array.

Output Format:

- If at least one subset exists whose sum equals the target, print:

Found a subset with given sum

- If no such subset exists, print:

No subset with given sum

Note: This question doesn't contain partial weightages, all test cases must be passed.

Source Code:

CTC79176.c

```
#include<stdio.h>

int subsetSum(int arr[], int n, int target, int index) {

    if (target == 0)
        return 1;

    if (index >= n || target < 0)
        return 0;

    if (subsetSum(arr, n, target - arr[index], index + 1))
        return 1;

    return subsetSum(arr, n, target, index + 1);
}

int main() {
    int target, n;

    scanf("%d", &target);
    scanf("%d", &n);

    int arr[n];
    for (int i = 0; i < n; i++) {
```



```

        scanf("%d", &arr[i]);
    }

    if (subsetSum(arr, n, target, 0))
        printf("Found a subset with given sum");
    else
        printf("No subset with given sum");

    return 0;
}

```

Execution Results - All test cases have succeeded!

Test Case - 1

User Output

9

3

5 4 2

Found a subset with given sum

Test Case - 2

User Output

87

5

12 23 21 10 17

No subset with given sum